Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- İşletim Sistemlerinin Yapıları
  - Bu ders “yapı” kavramını geniş anlamda ele alır: donanım yapıları, veri yapıları ve kavramlar.
  - İşletim sisteminin tarihsel gelişimi boyunca benimsenen mimariler ve yapılar (monolithic, microkernel, hybrid, modular).
- İşletim Sisteminin Sağladığı Servisler (Hatırlatma)
  - User Interface, Program Execution, I/O Operations, File System Manipulation, Communications, Error Detection, Resource Allocation, Accounting, Protection and Security.
- Kullanıcı ile İşletim Sistemi Arasındaki Arayüz
  - Command Line Interface (Shell/Kabuk): klavyeyle komut satırı üzerinden etkileşim. Unix ve türevlerinde Shell olarak adlandırılır. Script dili olarak da kullanılır (if, for, while yapıları, dallanmalar).
  - Grafik User Interface (GUI): İlk olarak Xerox firması tarafından geliştirilmiş; Steve Jobs Xerox’u ziyaret ederek bu konsepti Apple’a taşımış. MIT’de aynı dönemde X-Windows sistemi geliştirilmiş.
  - Touch Screen Interface: Dokunmatik ekran; mouse yerine parmak hareketleri (gesture) kullanılır.
- Sistem Çağrıları (System Calls)
  - İşletim sisteminin hizmet sunduğu en temel arayüz. Sistem çağrıları, donanım ile uygulama arasındaki sınırı belirler.
  - User mode (sistem çağrısının üstü) ve kernel mode (sistem çağrısının altı) ayrımı vardır.
  - Sistem çağrıları C ve C++ ile yazılır; iki parçadan oluşur: çağrının kendisi (eylem) ve kütüphanedeki fonksiyon/metod (çağrılabilmesini sağlayan).
- Standart API’ler
  - Win32 API (Windows), POSIX API (Unix/Linux/macOS), Java API (JVM üzerinden).
  - POSIX = Portable Operating System Interface; bir standarttır.
- Sistem Çağrılarının Çalışma Mekanizması
  - Uygulama bir kütüphane fonksiyonu çağırır (örn. printf → write sistem çağrısı). Kütüphane parametreleri alır, sistem çağrısı formatına dönüştürür.
  - Software interrupt (syscall, int instruction) oluşturulur; kontrol işletim sistemine geçer. İşletim sistemi işlemi yapar, iret ile kütüphaneye döner, kütüphane normal return ile uygulamaya döner.
  - Kernel güncellenirse libc/libc++ da güncellenmelidir (aradaki geçişi sağlarlar).
- Sistem Çağrılarının Tipleri (Kategorileri)
  - Process Control: end, abort, load, execute, create/terminate process, wait, signal, allocate/free memory.
  - File Management: create, delete, open, close, read, write, reposition, get/set attributes.
  - Device Management: request, release, read, write, attach, detach.
  - Information Maintenance: get/set time/date, get/set system data, get/set process attributes.
  - Communications: create/delete communication connection, send/receive messages, transfer status.

- Sistem Çağrılarına Parametre Aktarım Yöntemleri
  - En basit: register'lar üzerinden. Sınır: register sayısı.
  - Daha iyi: parametreler bellekte bir bloğa konur, bloğun adresi register'da geçirilir.
  - En esnek (kernel seçimli): OS uygun gördüğü yöntemi kullanır.
- İşletim Sistemi Yapıları
  - Basit Yapı (MS-DOS): kernel ve sistem programları sınırları belirsiz, monolithic.
  - Daha İyi Yapı (UNIX): kernel modülleri (file subsystem, process control, memory management vb.), ancak yine sınırlar belirsiz.
  - Layered Approach: Her katman sadece altındaki katmanın fonksiyonlarını kullanır. Avantaj: yapı ve hata ayıklama kolaylığı. Dezavantaj: katmanların tanımlanması zor, verim düşer.
  - Microkernel: Kernel minimumda tutulur; sadece temel hizmetler (process management, memory management, iletişim). Diğer servisler (file system, device driver) user mode'da çalışır. Avantaj: genişletilebilirlik, güvenlik, taşınabilirlik. Dezavantaj: user-kernel geçişlerinden performans kaybı.
  - Modular Approach: Modern işletim sistemleri (Solaris, Linux, macOS) çekirdeğin modüller halinde yüklenmesine izin verir. Microkernel ile monolitik arasında orta yol.

Hocanın Özellikle Vurguladığı Kısımlar

- Sistem Çağrılarının Önemi
  - Hoca özellikle vurgular: "Sistem çağrılarını aklınızın bir köşesinde tutun." Bütün programlar işletim sisteminden hizmet alırken bu arayüzden geçmek zorundadır. Modern OS'lerde başka yol yoktur.
- System Call ile API Ayrımı
  - API (kütüphane fonksiyonu) ile sistem çağrısı (kernel'daki implementasyon) farklı kavramlardır. Uygulama geliştirici API ile etkileşir; asıl işi kernel yapar. printf → write sistem çağrısına dönüşür.
- Unlink ve Process İlişkisi
  - Hoca vurgular: Bir process (örn. Excel) bittiğinde sistemden tamamen çıkmaz; çünkü kendisini oluşturan parent process (Explorer.exe) hâlâ çalışıyordur. Bu yüzden sistemde "ölü" process'ler kısa süreyle bulunabilir; bu durum normaldir.
- Protection ve Security Ayrımı
  - Hoca, protection ile security'nin farklı kavramlar olduğunu vurgular: Protection = koruma (process'in kendi sınırları içinde kalmasını sağlama); Security = dış tehditlere karşı koruma (kullanıcı kimlik doğrulama, yetkilendirme, bütünlük).
  - Protection işletim sisteminin temel görevidir; security daha çok bilişim güvenliği dersinin konusudur.
- Tüm Programlar Neden "Ödev" Olarak Değerlendirilir
  - Hoca, işletim sistemi dersinin ödevlerini özellikle vurgular: "Milyon tane zilyon tane soru sorulan bir ders"tir. Kopyala-yapıştır yapılamaz; kaynak araştırması, başkalarının kodlarına bakma teşvik edilir, ancak körü körüne kopyalama cezalandırılır (MOSS gibi benzerlik tespit araçları).

Kısa Tekrar Notları

- Sistem çağrıları = uygulama ile kernel arasındaki sınır.
- User mode vs kernel mode: sistem çağrısının üstü vs altı.
- POSIX standardı: Unix/Linux için taşınabilir sistem çağrısı standardı.
- 5 temel OS yapısı: Simple (MS-DOS), Monolithic (UNIX), Layered, Microkernel, Modular.
- Sistem çağrısı tipleri: Process Control, File Management, Device Management, Information Maintenance, Communications.
- Protection = process'in sınırlar içinde kalması; Security = dış tehdit koruması.
- Aynı kullanıcının iki process'i bile doğrudan birbirinin belleğine erişemez (koruma).

Detaylı Açıklamalar

Ders 4, işletim sistemi yapılarını, sistem çağrılarını ve OS bileşenlerini detaylı şekilde ele alır. "Yapı" kavramı geniş anlamda ele alınır: donanım yapıları, veri yapıları, kavramlar. Geçen

haftalardan hatırlatma yapılarak başlanır: OS servisleri, process kavramı, donanım-yazılım ilişkisi.

Kullanıcı arayüzleri (UI) detaylı şekilde anlatılır. Command Line Interface (Shell), 1960-70'lerde kullanıcıların bilgisayarla etkileşimini sağlayan temel yöntemdi. Shell bir programlama dili olarak da kullanılabilir (if/else, for, while, değişkenler, fonksiyonlar). GUI'nin tarihi Xerox firması ile başlar; SteveJobs'un Xerox'u ziyareti ve MIT'de geliştirilen X-Windows sistemi anlatılır. Modern dokunmatik arayüzler (gesture) ise mouse yerine parmak hareketlerine dayanır.

Sistem çağrıları (System Calls) dersin ana konusudur. Sistem çağrıları, uygulama programlarının işletim sistemi hizmetlerine eriştiği arayüzdür. Modern OS'lerde user mode (sınırlı yetki) ve kernel mode (tam yetki) ayrımı vardır. Sistem çağrıları bu iki mod arasındaki geçişi sağlar. Sistem çağrıları genellikle C/C++ ile yazılır ve iki parçadan oluşur: çağrının kendisi (kernel'da) ve kütüphanedeki fonksiyon (çağrılabilmesi için).

Sistem çağrılarının çalışma mekanizması detaylı şekilde anlatılır: Uygulama bir API fonksiyonu (örn. printf) çağırır. Kütüphane fonksiyonu parametreleri alır, sistem çağrısı formatına dönüştürür, software interrupt (syscall/int instruction) oluşturur. İşletim sistemi bu interrupt'ı yakalar, gerekli işlemi yapar, sonucu döndürür. Kütüphane normal return ile uygulamaya döner. Bu nedenle kernel güncellendiğinde libc/libc++ da güncellenmelidir; aradaki bağlantıyı bunlar sağlar.

Sistem çağrıları beş ana kategoride sınıflandırılır: Process Control (fork, exec, wait, exit, kill, signal), File Management (open, close, read, write, lseek, stat), Device Management (ioctl, read, write), Information Maintenance (getpid, gettimeofday, setrlimit), Communications (pipe, socket, send, recv, bind, listen, accept).

İşletim sistemi yapıları (OS Structures) bölümünde farklı yaklaşımlar karşılaştırılır: Monolithic (MS-DOS, klasik UNIX), Layered (her katman sadece altındakini kullanır), Microkernel (sadece temel hizmetler kernel'da, diğerleri user mode'da) ve Modular (modern yaklaşım, yüklenebilir modüller). Her yapının avantaj ve dezavantajları tartışılır. Microkernel'in genişletilebilirlik, güvenlik ve taşınabilirlik açısından avantajlı olduğu, ancak user-kernel geçişlerinin performans kaybına yol açtığı vurgulanır.

Hoca, OS hizmetlerinin nasıl sunulduğunu şekilsel olarak gösterir: Hardware → Operating System (services) → System Calls (arayüz) → Kullanıcı. Bu yapı modern tüm OS'lerde aynıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.